Ders 4 Genişletilmiş Notu: Move-To-Front Kodlama, Prefix Kodlar, Shannon-Fano ve Huffman Kodlamaya Giriş

Ders 4 Genişletilmiş Notu: Move-To-Front Kodlama, Prefix Kodlar, Shannon-Fano ve Huffman Kodlamaya Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geçen Dersin Özeti: Entropi ve Artıklık
- 2. İstatistiksel Yöntemlere Giriş ve Ön İşlem Fikri
- 3. Move-To-Front Kodlama
- 4. Kelime Bazında Move-To-Front
- 5. Prefix Kod Kavramı
- 6. Değişken Uzunluklu Kodlar ve Artıklık
- 7. Prefix Olan ve Olmayan Kod: Code1 ve Code2
- 8. Logaritmik Prefix Kod
- 9. Unary Kod ve Start-Step-Stop Kodları
- 10. C1, C2, C3 ve C4 Kodları
- 11. Golomb Kodu
- 12. Statik, Yarı Statik ve Adaptif Yöntemler
- 13. Shannon-Fano Kodlama
- 14. Huffman Kodlamaya Giriş
- Örnek Sorular

1. Geçen Dersin Özeti: Entropi ve Artıklık

Hoca derse, geçen hafta bitirilen bilgi teorisini hatırlatan bir soruyla başladı: entropi veri sıkıştırmada neyi ifade eder?

Soru: Entropi veri sıkıştırmada ne ifade eder? **Cevap:** Bir öğrenci entropinin bir olayın meydana gelme olasılığıyla o olayın taşıdığı bilgi arasındaki ilişki olduğunu söyledi. Hoca bunu tamamladı: bilgi doğrudan ölçülebilen bir şey olmadığı için olasılıkla ölçülebilir hale getirilir. Bilgi $\log_2 \frac{1}{P}$'dir; bunu olasılıkla çarpıp toplarken P yukarı alınır ve eksi işaret eklenir:

$$H = -\sum_i P_i \log_2 P_i$$

Entropinin sonucu, sıkıştırılacak bir metnin bir sembolünün (karakterinin) ortalama kaç bitle kodlanabileceğini verir.

İkinci kavram artıklıktır (redundancy). Bir yöntem seçilip sıkıştırma yapıldığında elde bir ortalama mesaj uzunluğu (S, sembol başına ortalama bit) ve bir de entropi vardır. Artıklık bu ikisi arasındaki farktır. Artıklık ne kadar küçükse o kadar ideal bir yöntem seçilmiş demektir. Artıklığı küçültmek için ya farklı yöntemler denenir ya da verinin fiziksel yapısı hakkında bilgi edinilip ona göre sıkıştırılır.

Hoca geçen haftaki iki fikri de hatırlattı: tekrar eden örüntüleri blok olarak almak daha iyi sıkıştırma performansı veriyordu; semboller arasındaki farkları almak (residual dizi) birbirine çok yakın sayılardan oluşan bir dizi üretiyor ve bu semboller 7-8 bit yerine 2-3 bitle kodlanabiliyordu.

2. İstatistiksel Yöntemlere Giriş ve Ön İşlem Fikri

Dersin başında yöntemler istatistiksel ve sözlüksel olarak iki başlığa ayrılmıştı. Hoca istatistiksel yöntemlerin altında yüzlerce yöntem olduğunu, her kullanım alanından bazılarını seçerek anlatacağını söyledi. İlkelden gelişmişe doğru gidilecek; amaç nereden nereye gelindiğini görmektir.

Bazı olasılık yöntemlerinde, olasılık kısmı kullanılmadan önce veriye bir ön işlem uygulanır. Bunlardan biri Move-To-Front kodlamadır.

3. Move-To-Front Kodlama

3.1 Tanım

Slayttaki tanım şöyledir (ekrandan, 00:10:00):

- Bölgesel adaptif bir entropi kodlama yöntemidir.
- Eldeki mevcut alfabe kullanılarak input stream'de yer alan karakterlere kullanım sıklıklarına bağlı olarak bir kod verilir.
- Yöntem gereği sıklıklar önceden hesaplanmaz.
- Sıkıştırma performansını artırmak için öncesinde kullanılan bir kodlamadır.

Adı üstünde, bir şey bir yerden bir yere, alfabenin önüne doğru taşınır. Adaptif olması, sürekli bir değişkenlik göstermesidir. Alfabe, üzerinde işlem yapılan girdinin (metin, görüntü, ne olursa) içindeki sembollerin kümesidir. "Sıklıklar önceden hesaplanmaz" ifadesi, girdinin başında "şu sembolden şu kadar var" diye bir tahmin yapılmadığını, girdinin teker teker elden geçirildiğini anlatır.

3.2 Karakter Bazında Örnek

Örnekte (ekrandan, 00:20:00) input stream `a b c d d c b a m n o p p o n m`, alfabe `{abcdmnop}` (8 karakter). Hoca karakterlerin yan yana anlamlı bir şey oluşturmasının önemli olmadığını, kolay anlatmak için seçildiğini söyledi. Alfabedeki arama 0. indisten başlar: a 0, b 1, c 2, d 3, m 4, n 5, o 6, p 7.

Kural: sıradaki sembolün alfabedeki indisi kod olarak yazılır, sonra sembol alfabenin en önüne taşınır, diğerleri kaydırılır.

```
Input  Alfabe (önce)  Kod
a      abcdmnop       0
b      abcdmnop       1
c      bacdmnop       2
d      cbadmnop       3
d      dcbamnop       0
c      dcbamnop       1
b      cdbamnop       2
a      bcdamnop       3
m      abcdmnop       4
n      mabcdnop       5
o      nmabcdop       6
p      onmabcdp       7
p      ponmabcd       0
o      ponmabcd       1
n      opnmabcd       2
m      nopmabcd       3
son:   mnopabcd
```

Kod dizisi `0 1 2 3 0 1 2 3 4 5 6 7 0 1 2 3` olur. Bundan sonra yapılacak şey bu kod kolonundan istatistik çıkarıp bir istatistiksel yöntemle sıkıştırmaktır. 16 sembol vardır; 0, 1, 2 ve 3 üçer kez geçer ($P = 3/16$), 4, 5, 6 ve 7 birer kez geçer ($P = 1/16$). Entropi:

$$H = 4 \cdot \frac{3}{16} \log_2 \frac{16}{3} + 4 \cdot \frac{1}{16} \log_2 16 \approx 2{,}81 \text{ bit/karakter}$$

(slayt ve python ile doğrulandı). Yani bu olasılıklarla herhangi bir sıkıştırma yöntemi kullanıldığında bir karakter ortalama 2,81 bitle kodlanır; tek tek bakıldığında bazı karakterler 1, 2 ya da 3-5 bit alabilir ama ortalama buna gelir. Ön işlem yapılmasaydı her karakter için 8 bit kullanılacaktı. Slaytta, kod içindeki değerleri kodlamak için Huffman ya da aritmetik kodlama kullanıldığı yazıyor (ekrandan, 00:20:00).

3.3 Kod Çözme

Kod dizisi sıkıştırılır, karşı tarafta açılır ve kod dizisi yeniden elde edilir. Amaç bundan özgün input stream'e dönmektir. Slaytta "Kod çözme işleminde input tersten okunur ve alfabenin ilk hali oluşturulur" yazıyor (ekrandan, 00:20:00). Hoca alfabenin son halini (`mnopabcd`) alıp kodları uyguladı: 0 → m, başa taşınır; 1 → n, başa taşınır; 2 → o; 3 → p; 0 → p; 1 → o; … en son 3 → a. Elde edilen dizi `m n o p p o n m a b c d d c b a` sondan başa okununca özgün input `a b c d d c b a m n o p p o n m` çıkar, alfabe de ilk haline (`abcdmnop`) döner.

> **Ders sonrası not:** Hocanın gösterdiği yol (son alfabeden başlayıp kodları baştan sona uygulamak ve sonucu ters çevirmek) bu örnekte doğru sonuç veriyor, çünkü örnek input simetriktir; farklı bir girdide çalışmaz (python ile denendi). Genel kod çözme iki biçimde yapılır: (1) alfabenin ilk haliyle başlayıp kodlar baştan sona işlenir, indisteki sembol yazılır ve başa taşınır; (2) alfabenin son haliyle başlayıp kodlar sondan başa işlenir, alfabenin başındaki sembol yazılır ve kodun gösterdiği indise geri konur, sonra çıkan dizi ters çevrilir. İki yol da bu örnekte `abcddcbamnopponm` verir.

Hoca özetle şunu söyledi: veriye önce bir dönüşüm (transform) uygulandı, dönüştürülmüş veri kodlandı, açıldı ve MTF'nin tersi uygulanarak özgün veri elde edildi. Girdi doğrudan kodlansaydı sıkıştırma için biraz daha fazla harcama gerekebilirdi.

MTF'nin farklı versiyonları vardır. Örneğin bir k parametresi kullanılabilir (move-to-front-k); k 0 alınabileceği gibi 1, 2, 3 gibi değerler de alabilir. Hoca genelde işlenen sembolü en öne getiren temel versiyonu tercih ettiğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Literatürdeki bu tür versiyonlarda büyük olasılıkla sembol en öne değil, bulunduğu yerden k konum öne taşınır ya da ancak belirli sayıda görüldükten sonra öne alınır. Amaç, bir kez görülen nadir bir sembolün sık sembolleri arkaya itmesini önlemektir.

3.4 Aynı Girdi MTF'siz

Aynı input için yalnız alfabedeki yerlere bakılır, hiçbir sembol öne taşınmaz (ekrandan, 00:22:30). Kod dizisi `0 1 2 3 3 2 1 0 4 5 6 7 7 6 5 4` olur. Her indis iki kez geçer, 8 eşit olasılıklı sembolün entropisi $H = 3$ bittir.

2,81 ile 3 arasında fark yok gibi görünebilir; pratikte 2,81 bit de olmayacağı için 3 bit olabilir. Hoca neden uğraşıldığını şöyle açıkladı: alfabe burada yalnız 8 karakterlidir. Gerçek uygulamalarda alfabe çok daha büyüktür; karakter değil kelime kodlanırsa kelime sayısı on binlercedir ve MTF orada çok daha işe yarar. Küçük alfabede fark az görünse de MTF'siz entropi daha yüksek çıkmıştır.

MTF'siz kod çözme de aynıdır, alfabe hiç değişmediği için her kodun gösterdiği sembol yazılır.

4. Kelime Bazında Move-To-Front

MTF'de yalnız karakterler değil kelimeler de kodlanabilir (ekrandan, 00:30:00). Kelimelerde elde hazır bir alfabe olmadığı için farklı davranılır; alfabe yerine sözlük denir. String: `the boy on my right is the right boy`.

Sözlük boş başlar. Gelen kelime sözlükte yoksa sözlüğün sonuna eklenir; kodu olarak indisiyle birlikte kelimenin kendisi yazılır. Kelime sözlükte varsa yalnız indisi yazılır. Her iki durumda da kelime sözlüğün başına taşınır.

Kelime	Sözlük (önce)	Sözlük (sonra)	Kod
the	()	(the)	0the
boy	(the)	(the, boy)	1boy
on	(boy, the)	(boy, the, on)	2on
my	(on, boy, the)	(on, boy, the, my)	3my
right	(my, on, boy, the)	(my, on, boy, the, right)	4right
is	(right, my, on, boy, the)	(right, my, on, boy, the, is)	5is
the	(is, right, my, on, boy, the)	aynı	5
right	(the, is, right, my, on, boy)	aynı	2
boy	(right, the, is, my, on, boy)	aynı	5
son:	(boy, right, the, is, my, on)		

İkinci `the` geldiğinde kelime sözlükte zaten vardır, sona eklenmez; 5. indiste olduğu için yalnız `5` yazılır. Hoca, sözlükte olmayan bir kelime (ör. `girl`) gelseydi sözlüğün sonuna (6. indis) ekleneceğini, `6girl` yazılacağını ve `girl`'ün başa taşınacağını söyledi.

Sıkıştırma kod kolonuna uygulanır: bu çıktı bir sıkıştırma yöntemine gönderilir, karşı taraf açar ve kod kolonunu elde eder. Sonra kod çözme yapılır: sözlük yine boş başlar. `0the` 0. indise `the` koy ve yaz demektir; `the` başa taşınır. `1boy`, `2on`, `3my`, `4right`, `5is` aynı şekilde işlenir. Sonra yalnız sayılar gelir: `5` sözlüğün 5. indisindeki kelime yani `the`; `2` → `right`; `5` → `boy`. Her yazılan kelime sözlüğün başına taşınır ve çıktı özgün metni verir.

Hoca MTF'yi, sıkıştırmadan önce veri üzerinde bir ön işlem yapılarak verinin daha iyi sıkıştırılıp sıkıştırılamayacağı sorusuna verilmiş bir cevap olarak özetledi. Benzer bir ön işlem yöntemi de Burrows-Wheeler Transform'dur (BWT); adı iki araştırmacının isminden gelir ve veriye MTF'deki gibi bir dönüşüm uygular.

5. Prefix Kod Kavramı

Hoca bunu çok önemli bir kavram olarak tanıttı: Shannon kodu, Huffman kodu, aritmetik kod ve sonra görülecek yöntemlerin hepsi prefix özelliğine sahiptir. Olasılık yöntemleri kullanılıyorsa üretilen kodların hepsi prefix olmak zorundadır. Prefix olmayan kod çözülemez.

Örnek: A'ya `1`, B'ye `11` kodu verilsin. `baba` kodlanırsa B = `11`, A = `1`, B = `11`, A = `1` ve altı tane 1 elde edilir. Bunu açarken her 1'i ayrı alırsanız `AAAAAA`, ikişer alırsanız `BBB` çıkar; `baba` belki de açılır, ama hangisinin seçileceği keyfe kalmıştır. Hoca "aslında keyif diye bir şey yok" dedi: önemli olan kodların prefix özelliğine sahip olmasıdır. Bu iki kod prefix değildir.

6. Değişken Uzunluklu Kodlar ve Artıklık

ASCII tablosunda bütün sembollere 8 bit verildiği için bunlar sabit uzunluklu kodlardır. Semboller farklı uzunlukta bitlerle ifade edilirse değişken uzunluklu kod olur (ekrandan, 00:40:00).

6.1 Eşit Olasılıklar

$\{a_1, a_2, a_3, a_4\}$ hepsi eşit olasılıklı olsun ($P_i = 0{,}25$):

$$H = -(4 \cdot 0{,}25 \log_2 0{,}25) = 2 \text{ bit}$$

4 sembol $2^n = 4$ eşitliğinden $n = 2$ bitle `00, 01, 10, 11` diye kodlanır; kodlar birbirinden farklıdır ve prefix özelliği de vardır. $S = 2$, $H = 2$, $R = |H - S| = 0$: ideal kodlama.

6.2 Farklı Olasılıklar

Olasılıklar değişsin: $P(a_1) = 0{,}49$, $P(a_2) = 0{,}25$, $P(a_3) = 0{,}25$, $P(a_4) = 0{,}01$ (ekrandan, 00:40:00). Artık hepsini yine 2 bitle kodlamak doğru olmaz. Entropinin söylediği şudur: olasılığı yüksek bir olayın taşıdığı bilgi küçüktür ve daha az bitle ifade edilmelidir. Olasılığı çok

düşük bir olayın başa gelmesi önemli bir olaydır, taşıdığı bilgi büyüktür ve daha çok bitle ifade edilmelidir. Buna göre a_1'in bit sayısı diğerlerinden az, a_4'ünki diğerlerinden fazla olmalıdır.

Entropi $H = 1{,}57$ bit çıkar (python ile doğrulandı). Eşit olasılıkta 2 olan entropi 1,57'ye inmiştir; yani ortalamada 2 bitten daha azıyla kodlama yapılabilir. Yöntem bilinmese de her mühendis 4 sembolü 2 bitle yazabilir; o zaman $S = 2$, $H = 1{,}57$ ve $R = 0{,}43$ olur. Artıklığın sıfır olmaması, daha uygun bir yöntemle daha iyi sıkıştırma yapılabileceğini gösterir.

Hoca sıkıştırma gözüyle de aynı şeyi söyledi: karşıya daha çok gelen bir şeyi daha az bitle ifade edersem daha çok şey sıkıştırmış olurum.

7. Prefix Olan ve Olmayan Kod: Code1 ve Code2

Henüz bir yöntem bilinmiyor, yalnız "olasılığı yüksekse az bit" kuralı biliniyor. Hoca bir öğrencinin bu sembollere 1, 2, 3, 3 bitlik kodlar verdiğini, bir başkasının farklı bir kod (Code2) yazdığını, bir başkasının başka bir kod üretebileceğini anlattı (ekrandan, 00:50:00):

Sembol	Olasılık	Code1	Code2
a1	0,49	1	1
a2	0,25	01	01
a3	0,25	010	000
a4	0,02	001	001

Input `a1 a3 a2 a1 a3 a3 a4 a2 a1 a1 a2 a2 a1 a1 a3 a1 a1 a2 a3 a1`: 9 tane a_1, 5 tane a_2, 5 tane a_3, 1 tane a_4, toplam 20 sembol. Code1 ile:

- 9 a_1 × 1 bit = 9
- 5 a_2 × 2 bit = 10
- 5 a_3 × 3 bit = 15
- 1 a_4 × 3 bit = 3

Toplam 37 bit. 8 bitle kodlansaydı $20 \times 8 = 160$ bit gerekirdi. Ortalama mesaj uzunluğu $S = 37/20 = 1{,}85$ bit, $H = 1{,}57$, $R = |H - S| = 0{,}28$. Slayt "sıkıştırma performansı iyi, ancak bu kodun geri dönüşü yoktur. Neden?" diye soruyor.

> **Ders sonrası not:** Slayttaki tabloda a_4'ün olasılığı 0,02 yazıyor (toplam 1,01 olur); önceki slaytta ve $H = 1{,}57$ hesabında 0,01 kullanılmış. Ayrıca bu girdideki sayımlardan (9/20, 5/20, 5/20, 1/20) entropi hesaplanırsa $H \approx 1{,}73$ bit ve $R \approx 0{,}11$ çıkar (python ile hesaplandı); slayttaki 0,28, tablodaki olasılıklara göredir.

Hoca a_3a_2 dizisini iki kodla kodlayıp çözdü:

- Code1: a_3 = `010`, a_2 = `01` → `01001`. Çözücü `0` için kod arar, yok; `01` var: a_2. Kalan `0` yok, `00` yok, `001` var: a_4. Sonuç a_2a_4; girdi a_3a_2 idi, yanlış açıldı.
- Code2: a_3 = `000`, a_2 = `01` → `00001`. `0` yok, `00` yok, `000` var: a_3. Kalan `0` yok, `01` var: a_2. Sonuç a_3a_2, doğru.

Fark şudur: Code1'de a_2'nin kodu `01`, a_3'ün kodu `010`'ın başlangıcıdır. Çözücü `01`'i görünce bunun a_2 mi, yoksa üçüncü bite bakılıp `010` mu olduğunu bilemez. Code2'de ise `1` ile başlayan başka kod, `01` ile başlayan başka kod, `000` ya da `001` ile başlayan başka kod yoktur.

Bir kodun prefix olabilmesi için gerekli ve yeterli şart: bir sembolün kodu başka bir sembolün kodunun başlangıcını oluşturmamalıdır. Slayttaki ifadeyle, bir sembolün sahip olduğu kodun bit pattern'ı diğer sembolün ön eki olamaz.

Bu şart bazı kodların kullanılamayacağını da söyler. Code2'de 1 bitlik `0` kodu kullanılmadı, çünkü 0 ile başlayan başka kodlar var. `00` da alınamaz, çünkü `00` ile başlayan kodlar var. Prefix kod kümesi çıkarılırken çıkarılabilecek bütün kodlar çıkmaz; kullanılırsa hataya götürecek kodlar elenir.

Soru: İnteger sayıların binary gösterimi prefix özelliğine uyar mı? **Cevap:** 0, 1, 10, 11, 100, 101... yazıldığında 1 ile başlayan birçok kod vardır, ileride 10 ile, 100 ile başlayan birçok kod olacaktır. Bu nedenle integer sayıların düz binary gösterimi prefix değildir.

Hoca, olasılık tabanlı yöntemlerin dışında statik olarak geliştirilmiş, formülle üretilen prefix kodlar da olduğunu söyleyip bunlara geçti. Bu kodların fikri şudur: biri bir formül ortaya atar,

o formülle yazılan bütün kodlar prefix olur; olasılıkları çıkarılmış bir dokümana bu hazır kodlar uydurulup sıkıştırma yapılabilir.

8. Logaritmik Prefix Kod

Slayttaki kural (ekrandan, 00:56:40): $i \geq 1$ olmak şartıyla, i. kodun binary değerinin başlangıcına $\lfloor \log_2 i \rfloor$ adet sıfır eklenir.

Hoca önce taban ve tavan işaretlerini sayı doğrusu üzerinde hatırlattı. $\lfloor 5{,}1 \rfloor$, 5,1'den küçük en büyük tam sayıdır: 5. $\lceil 5{,}1 \rceil$, 5,1'den büyük en küçük tam sayıdır: 6.

```
i   floor(log2 i)  kod         uzunluk
1   0              1           1
2   1              010         3
3   1              011         3
4   2              00100       5
5   2              00101       5
6   2              00110       5
7   2              00111       5
8   3              0001000     7
...
15  3              0001111     7
16  4              000010000   9
```

$i = 1$ için $\log_2 1 = 0$, sıfır eklenmez, kod `1`. $i = 2$ için $\log_2 2 = 1$, bir sıfır ve 2'nin binary'si: `010`. $i = 3$ için $\log_2 3$ bir küsurdur, tabanı 1'dir: `011`. $i = 4 = 2^2$ için iki sıfır: `00100`. 5, 6, 7 için $\log_2$ değeri 2 küsurdur, tabanı 2'dir. $i = 8 = 2^3$ için üç sıfır. Hiç `1` ile, `010` ile, `011` ile, `00100` ile başlayan başka kod yoktur; bunların hepsi hiç uğraşmadan elde edilmiş prefix kodlardır.

En yüksek olasılıklı sembol 1 bitle, ikinci ve üçüncü 3 bitle kodlanır. Uzunluk genel olarak $2\lfloor \log_2 i \rfloor + 1$'dir: 16 için $\log_2 16 = 4$, $2 \cdot 4 + 1 = 9$ bit; tablodaki 9 buna karşılık gelir. 16 prefix koda ihtiyaç varsa en az 1, en çok 9 bit kullanılır.

> **Ders sonrası not:** Hoca 64 prefix kod üretilirse en düşük olasılıklı sembolün 9 bitle kodlanacağını söyledi. Formüle göre $i = 64$ için 6 sıfır ve 7 bitlik binary, toplam $2 \cdot 6 + 1 = 13$ bit olur; 9 bit $i = 16$'nın uzunluğudur.

9. Unary Kod ve Start-Step-Stop Kodları

9.1 Unary Kod

Slayttaki tanım (ekrandan, 01:10:00 ve 01:20:00): negatif olmayan n integer sayısının unary kodu $(n-1)$ adet 1 ve onları takip eden bir tane 0'dan, ya da $(n-1)$ adet 0 ve onları takip eden bir tane 1'den oluşur.

```
n  code   alter code
1  0      1
2  10     01
3  110    001
4  1110   0001
5  11110  00001
```

$n = 1$ için $n - 1 = 0$ tane 1 yani hiç 1 yok, sadece 0. $n = 2$ için bir 1 ve 0. 1'ler ve 0'ların yer değiştirdiği alternatif kod da aynı işi görür. n değeri 0'dan başlıyorsa $n - 1$ değil n tane alınır: 0 için `0`, 1 için `10`, 2 için `110`.

9.2 Start-Step-Stop Tripleti

Unary kodlar "start-step-stop" kodu olarak da bilinir (ekrandan, 01:20:00). Start başlangıç noktası, step ne kadar aralıkla gidildiği, stop bitiş noktasıdır. Kurallar:

- $a = \text{start} + n \cdot \text{step}$
- n. kod kelimesi, n'in unary kodu ve arkasından a bitlik binary sayıdan oluşur.

- $a =$ stop olduğunda a bitinin önündeki 0 düşürülür. Düşürülmezse kod yine prefix olurdu, ama gereksiz bir bit harcanırdı.

Örnek: 680 adet unary kodu nasıl elde edilir? Triplet (3, 2, 9): start 3, step 2, stop 9. Bu üç sayı 680 farklı prefix kod için yeterlidir.

```
n  a=3+2n  n. kod kelimesi   kod sayısı    integer sınırı
0  3       0xxx              2^3 = 8       0-7
1  5       10xxxxx           2^5 = 32      8-39
2  7       110xxxxxxx        2^7 = 128     40-167
3  9       111xxxxxxxxx      2^9 = 512     168-679
```

- $n = 0$: $a = 3 + 0 \cdot 2 = 3$. 0'ın unary kodu (0'dan başladığı için n tane 1, yani hiç yok) `0`. Arkasından 3 bit gelir; 3 bitle 8 sayı yazılır: 0-7 arası 8 kod.
- $n = 1$: $a = 3 + 1 \cdot 2 = 5$. Unary `10`, arkasından 5 bit: $2^5 = 32$ sayı, 8-39 arası. Şimdiye kadar 40 kod.
- $n = 2$: $a = 7$. Unary `110`, arkasından 7 bit: 128 sayı, 40-167 arası. Toplam 168 kod.
- $n = 3$: $a = 3 + 3 \cdot 2 = 9 =$ stop. Unary kod `1110` olurdu, ama a stop'a eşit olduğu için sondaki 0 düşer: `111`. Arkasından 9 bit: 512 sayı, 168-679 arası. Toplam $167 + 512 = 679$, yani 0'dan 679'a 680 kod.

`0` ile, `10` ile, `110` ile başlayan başka grup olmadığı için hepsi prefix koddur. Son gruptaki 0 düşürülebilir, çünkü arkasından başka grup gelmez ve prefix özelliğini korumaya gerek kalmaz; dördüncü bir grup olsaydı bu 0 çıkarılamazdı. 512 sembolün her birinde bir bit kazanılır; aksi halde 512 bit boşuna harcanırdı.

Toplam kod sayısı adım adım gitmeden şu formülle bulunur:

$$\frac{2^{\text{stop}+\text{step}} - 2^{\text{start}}}{2^{\text{step}} - 1} = \frac{2^{11} - 2^3}{2^2 - 1} = \frac{2040}{3} = 680$$

(python ile doğrulandı). Örneğin 700 kod gerekiyorsa farklı tripletler denenip bu formülle kontrol edilerek uygun triplet aranır; tam 700 çıkmayabilir, 800 çıkabilir, 700'ü kullanılır. Hoca stop parametresine bağlı olarak kod sayısının üstel olarak arttığını söyledi.

10. C1, C2, C3 ve C4 Kodları

İnsanlar unary kod ve başka formüllerden yararlanarak farklı prefix kodlar üretmişlerdir. Hoca bunların özel bir adı olmadığını, slaytta C_1-C_4 diye adlandırıldığını söyledi (ekrandan, 01:25:20). Kullanılan gösterimler:

- n: elde edilmek istenen kodun sırası (ör. 10., 255., 652. prefix kod).
- $B(n)$: n sayısının binary karşılığı.
- $|B(n)|$: binary sayının dijit uzunluğu (1, 2, 2, 3, 3, 3, 3, 4, ...).
- $\bar{B}(n)$: en soldaki bit (her zaman 1) atıldığında kalan bitler. 1 için hiçbir şey, 2 için `0`, 3 için `1`, 4 için `00`.

Slayttaki tablodan bir kısım:

```
n   B(n)    B̄(n)   C1          C2          C3          C4
1   1               0           0           0           0
2   10      0       10|0        100         100|0       10|0
3   11      1       10|1        110         100|1       11|0
4   100     00      110|00      10100       110|00      10|100|0
5   101     01      110|01      10110       110|01      10|101|0
6   110     10      110|10      11100       110|10      10|110|0
7   111     11      110|11      11110       110|11      10|111|0
8   1000    000     1110|000    1010100     10100|000   11|1000|0
15  1111    111     1110|111    1111110     10100|111   11|1111|0
16  10000   0000    11110|0000  101010100   10110|0000  10|100|10000|0
```

10.1 C1

C_1 iki parçadan oluşur (ekrandan, 01:30:00): n'in binary karşılığı $B(n)$'in dijit sayısının unary karşılığı ile $B(n)$'in en soldaki biti atılmış hali $\bar{B}(n)$'nin birleşimidir:

$$C_1 = \text{unary}(|B(n)|) \mid \bar{B}(n)$$

$n = 16$: $B(16) = 10000$, $|B(16)| = 5$, $\bar{B}(16) = 0000$. 5'in unary kodu (n 1'den başladığı için $5-1 = 4$ tane 1 ve 0) `11110` ya da alternatifi `00001`. Sonuç `11110|0000` ya da `00001|0000`. Hiçbir kod başka bir kodun başlangıcı değildir.

Hoca öğrencilerden $C_1(6)$'yı bulmalarını istedi: $B(6) = 110$, dijit sayısı 3, unary(3) = `110`, en soldaki bit atılınca `10` kalır: $C_1(6) = 110|10$.

C_1'in uzunluğu $2\lfloor \log_2 n \rfloor + 1$'dir: 16 için $2 \cdot 4 + 1 = 9$ bit (5 + 4).

10.2 C2

C_2, C_1'in yeniden düzenlenmesidir: C_1'in birinci bölümündeki her bit sırayla alınırken her bitin arkasına ikinci bölümden bir bit yazılır. Önce C_1 hesaplanmak zorundadır. $C_1(16) = 11110|0000$ için birinci bölümden 1, ikinciden 0, birinciden 1, ikinciden 0... diye gidilir: `101010100`. Bit sayısı değişmediği için uzunluk C_1 ile aynıdır. Bunlar da prefix koddur.

10.3 C3

n'nin binary gösteriminin dijit sayısı alınır; bu sayının C_2 kodu bulunur ve arkasına $\bar{B}(n)$ eklenir (ekrandan, 01:34:50):

$$C_3 = C_2(|B(n)|) \mid \bar{B}(n)$$

$n = 15$: $B(15) = 1111$, $|B(15)| = 4$. $C_2(4) = 10100$. $\bar{B}(15) = 111$. $C_3(15) = 10100|111$.

C_3 için C_2'ye, C_2 için C_1'e ihtiyaç vardır; peş peşe birbirinden üretilmiş prefix kodlardır.

> **Ders sonrası not:** Derste C_3'ün uzunluk formülü üzerinden yapılan hesap karıştı. Tanıma göre uzunluk $|C_2(|B(n)|)| + |B(n)| - 1$'dir: $n = 16$ için $5+4 = 9$ bit, $n = 15$ için $5 + 3 = 8$ bit (python ile doğrulandı, slayttaki tabloyla uyumlu).

Hoca uzun kodların neden sorun olmadığını şöyle anlattı: tablonun en altındaki kodlar 14 bit kadar uzundur. Olasılığı yüksek, metinde çok geçen bir sembol için 8 yerine 1 bit kullanılır. Olasılığı çok düşük, belki metinde 2-3 kez geçen bir sembole 14 bit verilmesinin kaybı, üst taraftan elde edilen kazanç yanında önemsizdir.

10.4 C4

C_4 diğer kodlara bağımlı değildir, kendi içinde türeyerek gider; birden fazla bölümden oluşur ve n büyüdükçe bölüm sayısı artar (ekrandan, 01:40:00). Kurallar:

- İlk olarak hesaplanacak n'in binary gösterimi yazılır.
- Sol tarafına $|B(n)| - 1$ değerinin binary gösterimi yazılır.
- İşleme, yazılan değer iki dijit kalıncaya kadar devam edilir.
- En son adımda sağ tarafa 0 yazılır.

$C_4(16)$:

```
16 = 10000                  -> |10000
|B(16)| - 1 = 5 - 1 = 4     -> |100|10000
|B(4)|  - 1 = 3 - 1 = 2     -> 10|100|10000   (iki dijit, dur)
sağa 0                       -> 10|100|10000|0
```

$C_4(5)$: $5 = 101$, 3 dijit, $3 - 1 = 2 \rightarrow$ `10|101`; 2 iki dijittir, durulur; sağa 0: `10|101|0`.

10.5 Hangi Kod Seçilmeli?

Hoca kodları karşılaştırdı. $n = 4$ için C_1, C_2 ve C_3 5 bit, C_4 6 bittir; burada C_4 daha kötüdür. $n = 15$ için C_1 ve C_2 7 bit, C_3 8 bit, C_4 7 bittir; burada C_4, C_3'ten iyidir. C_1-C_4 her n için aynı bit sayısında kod üretmez. Olasılık dağılımı sıralanıp hangi sıradaki sembollerin daha sık geldiğine bakılarak bu kodlardan hangisinin seçileceğine karar verilir.

11. Golomb Kodu

Golomb kodu, adını ortaya atan kişiden alır; unary kodlama yönteminin özelliklerini kullanarak oluşturulan pratik bir prefix koddur (ekrandan, 01:50:00). Adımlar:

- Kodları oluşturulacak alfabedeki karakterler kullanım frekanslarına ya da olasılıklarına göre büyükten küçüğe sıralanır; çünkü üst sıralar daha az bit alır.
- x olarak adlandırılan karakterlere 0'dan başlayarak ardışık değerler atanır (0., 1., 2. karakter).
- İdeali 2'nin kuvveti (2, 4, 8, 16, 32) olan bir M değeri seçilir.
- Her x için x_M ve x_L hesaplanır:

$$x_M = \left\lfloor \frac{x}{M} \right\rfloor, \qquad x_L = x \bmod M, \qquad x = x_M \cdot M + x_L$$

- Golomb kodu, x_M'in unary kodu ile x_L'nin binary değerinin birleşimidir.

Örnek, $M = 4$. Hoca sıralanmış sembollere en sık geçen harflerden örnek verdi (a, e, k, l, m, n, ...).

x	xM	xL	Unary xM	Binary xL	Golomb
0	0	0	0	00	000
1	0	1	0	01	001
2	0	2	0	10	010
3	0	3	0	11	011
4	1	0	10	00	1000
5	1	1	10	01	1001
6	1	2	10	10	1010
7	1	3	10	11	1011
8	2	0	110	00	11000

x_M: $0/4$, $1/4$, $2/4$, $3/4$ hep 0 küsurdur, tabanı 0. $4/4 = 1$; 5, 6, 7 için de 1; $8/4 = 2$. x_L: 0, 1, 2, 3, 0, 1, 2, 3, 0. x_M 0'dan başladığı için unary kodunda $n - 1$ değil n alınır: 0 için `0`, 1 için `10`, 2 için `110`. x_L 2 bitlik binary ile yazılır. Üç sıfırla başlayan, `011` ile, `1011` ile başlayan başka kod yoktur; hepsi prefix'tir (python ile doğrulandı).

12. Statik, Yarı Statik ve Adaptif Yöntemler

Hoca, istatistiksel yöntemler zaten prefix kod üretiyorsa bu hazır kodlara ne gerek olduğunu sordu ve istatistiksel yöntemlerin üç sınıfını hatırlattı:

- **Statik:** kodlar önceden hazırlanır (unary kodlar, C_1-C_4, Golomb gibi). Eldeki sembollerin olasılıkları çıkarılır ve semboller bu hazır kodlara atanır.
- **Yarı statik (semi-adaptif):** önce metin okunur, olasılıklar çıkarılır, sonra bu olasılıklara göre kod üretilir. Metnin içeriğine göre prefix kod elde edildiği için çok daha iyi sıkıştırma yapılır. Hoca "yarı statik" yerine "semi-adaptif" demenin daha doğru olacağını söyledi.
- **Adaptif:** bir kelimenin ya da karakterin kodu metin içinde defalarca değişebilir; 3 bitken 5 bite çıkabilir, 5 bitken 4 bite inebilir. Kodlaması daha zor, karmaşık ve zaman alıcıdır, ama sıkıştırma performansı daha yüksektir.

Sıradaki yöntemler yarı statik (semi-adaptif), sonra adaptif yöntemlerdir.

13. Shannon-Fano Kodlama

13.1 Yöntem

Shannon-Fano, sabit ve değişken uzunluklu kodlardan bahsedilirken adı geçen, olasılıklar 2'nin negatif kuvvetleri cinsinden olunca iyi sonuç veren yöntemdir. Olasılık tabanlıdır; statik değil yarı dinamik (yarı statik) bir yöntemdir. Slayttaki maddeler (ekrandan, 02:10:00):

- Değişken uzunluklu bir kodlama yöntemidir; kodları prefix'tir.
- Olasılık veya kullanım sıklıkları bilinen n elemanlı bir küme ile başlanır.
- Semboller azalan sırada olasılıklarına ya da kullanım sıklıklarına göre sıralanır.
- Semboller kümesi aynı veya yakın olasılık değerlerine göre iki kümeye ayrılır: iki tarafta da toplam olasılık birbirine en yakın olacak şekilde.
- Bir kümedeki sembollerin hepsi 1 değerini alırken diğer kümedeki semboller 0 değerini alır.
- Her alt küme kendi içinde bölünmeye devam eder; bölünecek bir şey kalmayınca iş biter.
- Bölünmeler bir ağacın seviyelerini gösterir. Semboller aşağıdan yukarı doğru okunur.
- Sembollerin olasılık değerleri 2'nin negatif kuvvetleri cinsinden $(1/2, 1/4, 1/8, 1/16)$ olduğunda en iyi sonucu verir. Bu ancak teoride olur; bir metinde karakterlerin olasılıkları çıkarıldığında 2'nin kuvvetleri cinsinden olmayacaktır.

Üste 1, alta 0 verilmesi zorunlu değildir; tersi de yapılabilir. Önemli olan, bir kez seçilen kuralın bütün bölmelerde aynı uygulanmasıdır.

13.2 Birinci Örnek

Olasılıklar büyükten küçüğe: A 0,25; E 0,20; K 0,15; L 0,15; M 0,10; N 0,10; Z 0,05 (ekrandan, 02:20:00). Harfler hocanın sonradan verdiği isimlerdir.

İlk bölme nereden yapılmalı? İlk sembolden sonra bölünürse 0,25 / 0,75 olur, olmaz. İkinci sembolden sonra 0,45 / 0,55 olur. Üçüncüden sonra 0,60 / 0,40 olur. En dengeli olan ikinci sembolden sonraki bölmedir; eşit olmak zorunda değildir, olabilecek en iyi bölme seçilir. Üst gruba 1, alt gruba 0 verilir.

Olasılık	1. bölme	2. bölme	3. bölme	4. bölme	Kod
0,25	1	1			11
0,20	1	0			10
0,15	0	1	1		011
0,15	0	1	0		010
0,10	0	0	1		001
0,10	0	0	0	1	0001
0,05	0	0	0	0	0000

Üst grup (0,25 / 0,20) ikiye bölünür: 1 ve 0. Alt grup 0,30 / 0,25 olarak bölünür (yaklaşık dengeli). 0,15 / 0,15 tam ortadan bölünür. Alt kısım 0,10 / 0,15 olarak bölünür (0,20 / 0,05 dengesiz olurdu). Son kalan iki eleman 1 ve 0 alır.

Bu yapı bir ağaçtır: başlangıç kök, semboller yapraklardır. Kodlar: A `11`, E `10`, K `011`, L `010`, M `001`, N `0001`, Z `0000`. Değişken uzunlukludur. Prefix kontrolü: `11` ile, `10` ile, `011` ile, `010` ile, `001` ile başlayan başka kod yoktur; prefix koddur.

Olasılıklar metinde karakterlerin, resimde piksellerin (renk değerlerinin) olasılıklarıdır. İçeriğe göre olasılıklar değişeceği için bölme aralıkları da değişir; olasılıkların hepsi 0,15 olsaydı çok farklı bölünür, farklı kodlar çıkardı.

Entropi:

$$H = -(0{,}25 \log_2 0{,}25 + 0{,}20 \log_2 0{,}20 + 2 \cdot 0{,}15 \log_2 0{,}15 + 2 \cdot 0{,}10 \log_2 0{,}10 + 0{,}05 \log_2 0{,}05) \approx 2{,}67 \text{ bit}$$

Ortalama mesaj uzunluğu:

$$S = 0{,}25 \cdot 2 + 0{,}20 \cdot 2 + 2 \cdot 0{,}15 \cdot 3 + 0{,}10 \cdot 3 + 0{,}10 \cdot 4 + 0{,}05 \cdot 4 = 2{,}7 \text{ bit}$$

$R = |2{,}67 - 2{,}7| = 0{,}03$ (python ile doğrulandı). Artıklık sıfıra çok yakındır; bu olasılıklar için ideal bir kodlama yapılmıştır.

13.3 İkinci Örnek: Nereden Bölünmeli?

Hoca altı tane 0,15 ve bir tane 0,10 olasılıklı semboller yazdı (ekrandan, 02:23:40 ve 02:26:30). Kümülatif toplamlar 0,15, 0,30, 0,45, 0,60, 0,75, 0,90, 1,00'dir.

Soru: 60/40 mı almalıyım, 45/55 mi? **Cevap:** Bir öğrenci 45/55 dedi. Hoca onayladı: 60 ile 40 arasında 20 birim, 45 ile 55 arasında 10 birim fark vardır. Bölme ilk üç sembolden sonra yapılır; üstlere 1, altlara 0 verilir.

Hoca entropinin nereden bölünürse bölünsün hep aynı kalacağını, ama bölme yerinin kodları ve ortalama mesaj uzunluğunu değiştirebileceğini vurguladı. Üst grup (üç tane 0,15) 0,15 / 0,30 ya da 0,30 / 0,15 diye bölünebilir; algoritma bir kez yazılacağı için böyle bir eşitlikte hangisinin seçileceğine karar verilmelidir. Hoca kuralı "büyük olasılık yukarıda kalacak şekilde" koydu: 0,30 üstte (1), 0,15 altta (0). Alt grup 0,30 / 0,25 diye bölünür. Kalan ikililer de 1 ve 0 alır.

Tahtadaki kodlar (ekrandan, 02:26:30):

Olasılık	Kod
0,15	111
0,15	110
0,15	10
0,15	011
0,15	010
0,15	001
0,10	000

Altı sembolün olasılığı 0,15 olmasına rağmen biri 2 bitle, diğerleri 3 bitle ifade edilmiştir. Üst grup farklı bölünseydi (0,15 yukarıda, 0,30 aşağıda) 2 bitlik kod üstte, 3 bitlikler altta olacaktı. Sembollerin kod değerleri değişir, entropi değişmez, ortalama mesaj uzunluğu olasılıkların durumuna göre değişebilir. Amaç artıklığı en küçük yapan yöntemi seçmektir.

Derste bu örneğin H ve S değerleri hesaplanmadı. Python ile: $H \approx 2{,}80$ bit, $S = 2{,}85$ bit, $R \approx 0{,}05$.

13.4 Kod Çözme ve Shannon-Fano'nun Yeri

Kodlanmış metin, kodlamanın ters işlemi yapılarak açılır; hoca bunu Huffman'da daha iyi göstereceğini söyledi. Sıradaki olasılık yöntemi Huffman'dır. Ortada Huffman varken Shannon-Fano genelde kullanılmaz; Huffman daha iyi sıkıştırma performansı verir, ortalama mesaj uzunluğu entropiye daha yakın olduğu için artıklığı küçüktür.

Shannon-Fano'nun neden anlatıldığı sorusuna hoca şöyle cevap verdi: olasılık dağılımı 2'nin kuvvetleri cinsinden ise Huffman'la uğraşmaya gerek yoktur; Shannon-Fano en iyi başarıyı verir ve kodlaması her bakımdan daha basittir. Bu yüzden birçok yöntem vardır ve seçim amaca bağlıdır: yerden mi zamandan mı kazanılmak isteniyor, sıkıştırma mı açma mı daha kısa sürmeli, nasıl bir uygulama yapılacak.

14. Huffman Kodlamaya Giriş

14.1 Genel Özellikler

Slayttaki maddeler (ekrandan, 02:30:00):

- Veri sıkıştırmada tek başına veya bir başka yöntemin içinde yer alarak kullanılan bir algoritmadır.
- Shannon-Fano'ya benzer, ancak kodları ters olarak yukarıdan aşağıya doğru oluşturur.
- Huffman algoritmasında kodlama ağacı oluşturulurken iki farklı yol izlenir. Sembollerin kod değerleri farklı olsa da entropi aynı çıkar.

> **Hocanın vurgusu:** Huffman JPEG içinde, ZIP içinde, WinRAR içinde kullanılır; bazen önünde bir ön işlem (transform) yapılması gerekir. Hoca bunu defalarca söylediğini, dönem sonunda bu soruya cevap verilememesinin kendisini üzdüğünü belirtti.

Shannon-Fano'da kodlar aşağıdan yukarı oluşturulup okunuyordu. Huffman'da kodlar yukarıdan aşağı, yani kökten yapraklara doğru okunur. Hocaya göre tersi yapılır, yapraklardan köke okunursa prefix kod çıkmayabilir; bazen denk gelir ama başka bir uygulamada gelmeyebilir.

Başlangıç Shannon-Fano'dan farksızdır: semboller olasılıklarına göre büyükten küçüğe sıralanır. Ama olasılıklar eşit ağırlıklı parçalara bölünmez; farklı bağlantılar yapılarak yine bir ağaç oluşturulur.

14.2 Örnek ve İki Yol

Olasılıklar: $a_1 = 0{,}4$, $a_2 = 0{,}2$, $a_3 = 0{,}2$, $a_4 = 0{,}1$, $a_5 = 0{,}1$ (ekrandan, 02:30:00). Semboller büyükten küçüğe sıralı olsa da işlemler sağdan sola, en küçüklerden başlayarak yapılır:

- En küçük iki olasılık toplanır: $0{,}1 + 0{,}1 = 0{,}2$ (a_{45}).
- Toplam, sıralamayı bozmayacak şekilde yerleştirilir. Önceden aynı değerde orijinal bir olasılık varsa (burada 0,2) toplam ya onun önüne ya da arkasına konur. "İki farklı yol" budur. Hiç eşitlik oluşmazsa sorun yoktur.
- Algoritma bir kez yazıldığı için hangi yol seçildiyse bütün adımlarda aynı kural uygulanır.
- Toplama, tek değer (1,0) kalana kadar sürer. Sonra dallara 0 ve 1 verilir.

Birinci yolda (ekrandan, 02:30:00) $a_{45} = 0{,}2$ ile $a_3 = 0{,}2$ toplanıp $a_{345} = 0{,}4$ elde edilir, bu a_2 ile toplanıp $a_{2345} = 0{,}6$ olur, en son a_1 ile 1,0'a ulaşılır. Kodlar `0, 10, 111, 1101, 1100`:

$$S = 0{,}4 \cdot 1 + 0{,}2 \cdot 2 + 0{,}2 \cdot 3 + 0{,}1 \cdot 4 + 0{,}1 \cdot 4 = 2{,}2 \text{ bit/sembol}$$

İkinci yolda (ekrandan, 02:40:00) a_2 ile a_3 birleşip $a_{23} = 0{,}4$, $a_{45} = 0{,}2$ ile a_1 birleşip $a_{145} = 0{,}6$ olur; bunlar 1,0'da birleşir. Kodlar `11, 01, 00, 101, 100`:

$$S = 0{,}4 \cdot 2 + 0{,}2 \cdot 2 + 0{,}2 \cdot 2 + 0{,}1 \cdot 3 + 0{,}1 \cdot 3 = 2{,}2 \text{ bit/sembol}$$

İki yolda da ortalama mesaj uzunluğu 2,2'dir. Kodlar birbirinden farklıdır: birinci yolda 1 bitlik ve 4 bitlik kod vardır; ikincide 4 bitlik kod, ama 1 bitlik kod da yoktur. Olasılıklar aynı olduğu için entropi, ortalama mesaj uzunluğu ve artıklık da aynıdır. Entropi derste hesaplanmadı; python ile $H \approx 2{,}12$ bit, $R \approx 0{,}08$.

Hoca slayttaki ağaç çiziminin kitaplardaki gibi biraz karışık olduğunu, sonraki derste daha kolay anlaşılır bir şekilde çizeceğini söyledi. Neden iki yolla uğraşıldığı ve hangisinin ne zaman seçilmesi gerektiği sorusunun cevabını varyans konusuyla birlikte sonraki derse bıraktı.

> **Ek bilgi (derste anlatılmadı):** Huffman'da eşit olasılıklarda birleştirilmiş düğüm listede mümkün olduğunca yukarıya konursa kod uzunluklarının varyansı büyük olasılıkla küçülür. Uzunluklar birbirine yakın olunca kodlayıcının çıkış hızı daha düzgün olur ve tampon (buffer) ihtiyacı azalır. Bu örnekte ikinci yolun (2-3 bitlik kodlar) uzunluk varyansı 0,16, birinci yolunki (1-4 bitlik kodlar) 1,36'dır (python ile hesaplandı).

Örnek Sorular

Soru 1 (derste sözlü, 00:03:41)

Entropi veri sıkıştırmada neyi ifade eder?

Çözüm (derste verildi): Bilgi $I = \log_2 \frac{1}{P}$ ile ölçülür. Entropi $H = -\sum P_i \log_2 P_i$, sıkıştırılacak metinde bir sembolün ortalama kaç bitle kodlanabileceğini verir. Artıklık, kullanılan yöntemin ortalama mesaj uzunluğu ile entropi arasındaki farktır.

Soru 2 (ekrandan, 00:20:00)

Input stream `a b c d d c b a m n o p p o n m`, alfabe `{abcdmnop}`. Move-To-Front ile kod dizisini, alfabenin son halini ve kod dizisinin entropisini bulunuz. MTF kullanılmasaydı entropi ne olurdu?

Çözüm (derste verildi): Kod `0 1 2 3 0 1 2 3 4 5 6 7 0 1 2 3`, son alfabe `mnopabcd`, $H = 2{,}81$ bit/karakter. MTF'siz indis dizisi `0 1 2 3 3 2 1 0 4 5 6 7 7 6 5 4`, $H = 3$ bit.

Soru 3 (ekrandan, 00:30:00)

`the boy on my right is the right boy` metnini kelime bazında Move-To-Front ile kodlayınız.

Çözüm (derste verildi): `0the 1boy 2on 3my 4right 5is 5 2 5`. Son sözlük `(boy, right, the, is, my, on)`.

Soru 4 (ekrandan, 00:50:00)

Code1 = {1, 01, 010, 001} ve Code2 = {1, 01, 000, 001}. Code1 ile sıkıştırma performansı iyi, ancak bu kodun geri dönüşü yoktur. Neden?

Çözüm (derste verildi): Code1 prefix değildir: a_2'nin kodu `01`, a_3'ün kodu `010`'ın ön ekidir. a_3a_2 = `01001` dizisi `01|001` = a_2a_4 diye açılır. Code2 prefix'tir ve `000|01` = a_3a_2 doğru açılır.

Soru 5 (derste sözlü, 00:51:07)

İnteger sayıların binary gösterimi prefix kod özelliğine uyar mı?

Çözüm (derste verildi): Uymaz; örneğin `1`, `10`, `11`, `100` gibi kodlarda `1` başka kodların ön ekidir.

Soru 6 (ekrandan, 01:20:00)

680 adet unary kodu nasıl elde edilebilir? (3, 2, 9) tripleti için her gruptaki kod sayısını ve integer sınırlarını bulunuz.

Çözüm (derste verildi): $a = 3 + 2n$: $n = 0 \rightarrow$ `0xxx`, 8 kod, 0-7; $n = 1 \rightarrow$ `10xxxxx`, 32 kod, 8-39; $n = 2 \rightarrow$ `110` + 7 bit, 128 kod, 40-167; $n = 3 \rightarrow a = 9 =$ stop, `111` + 9 bit, 512 kod, 168-679. Toplam $\frac{2^{11}-2^3}{2^2-1} = 680$.

Soru 7 (derste sözlü, 01:30:36)

$C_1(6)$ kodunu bulunuz.

Çözüm (derste verildi): $B(6) = 110$, $|B(6)| = 3$, unary(3) = `110`, $\bar{B}(6) = 10$. $C_1(6) = 110|10$.

Soru 8 (ekrandan, 01:34:50)

$C_3(15)$ kodunu bulunuz.

Çözüm (derste verildi): $|B(15)| = 4$, $C_2(4) = 10100$, $\bar{B}(15) = 111$. $C_3(15) = 10100|111$.

Soru 9 (ekrandan, 01:40:00; derste sözlü, 01:40:16)

$C_4(16)$ ve $C_4(5)$ kodlarını bulunuz.

Çözüm (derste verildi): $C_4(16) = 10|100|10000|0$; $C_4(5) = 10|101|0$.

Soru 10 (ekrandan, 01:50:00)

$M = 4$ için $x = 0, \ldots, 8$ değerlerinin Golomb kodlarını bulunuz.

Çözüm (derste verildi): `000, 001, 010, 011, 1000, 1001, 1010, 1011, 11000`.

Soru 11 (ekrandan, 02:20:00)

Olasılıkları 0,25; 0,20; 0,15; 0,15; 0,10; 0,10; 0,05 olan sembolleri Shannon-Fano ile kodlayınız; entropiyi, ortalama mesaj uzunluğunu ve artıklığı hesaplayınız.

Çözüm (derste verildi): Kodlar `11, 10, 011, 010, 001, 0001, 0000`. $H = 2{,}67$ bit, $S = 2{,}7$ bit, $R = 0{,}03$.

Soru 12 (ekrandan, 02:23:40; derste sözlü, 02:24:15)

Altı tane 0,15 ve bir tane 0,10 olasılıklı sembolü Shannon-Fano ile kodlarken ilk bölme 60/40 mı, 45/55 mi yapılmalıdır? Kodları bulunuz.

Çözüm (derste verildi): 45/55, çünkü iki taraf arasındaki fark daha küçüktür (10 birim). Büyük olasılık yukarıda kalacak kuralıyla kodlar `111, 110, 10, 011, 010, 001, 000`.

> **Çözüm (derste verilmedi):** $H = -(6 \cdot 0{,}15 \log_2 0{,}15 + 0{,}10 \log_2 0{,}10) \approx 2{,}80$ bit, $S = 5 \cdot 0{,}15 \cdot 3 + 0{,}15 \cdot 2 + 0{,}10 \cdot 3 = 2{,}85$ bit, $R \approx 0{,}05$ (python ile hesaplandı).

Soru 13 (ekrandan, 02:30:00 ve 02:40:00)

Olasılıkları 0,4; 0,2; 0,2; 0,1; 0,1 olan beş sembolü Huffman ile iki farklı yoldan kodlayınız ve ortalama mesaj uzunluklarını karşılaştırınız.

Çözüm (derste verildi): Birinci yol `0, 10, 111, 1101, 1100`, ikinci yol `11, 01, 00, 101, 100`; ikisinde de $S = 2{,}2$ bit/sembol. Kodlar farklı, entropi ve ortalama uzunluk aynıdır.

> **Çözüm (derste verilmedi):** $H = -(0{,}4 \log_2 0{,}4 + 2 \cdot 0{,}2 \log_2 0{,}2 + 2 \cdot 0{,}1 \log_2 0{,}1) \approx 2{,}12$ bit, iki yolda da $R \approx 0{,}08$ (python ile hesaplandı).

Dersin Toparlaması

Ders, entropi ve artıklığın hatırlatılmasıyla başladı ve istatistiksel yöntemlere hazırlık niteliğindeki ön işlem fikriyle devam etti. Move-To-Front kodlamada sembolün indisi yazılıp sembol alfabenin başına taşınır; tekrarlayan yerel örüntüler küçük indislere dönüşür ve örnekte entropi 3 bitten 2,81 bite düştü. Aynı fikir kelime bazında, boş başlayıp büyüyen bir sözlükle de uygulandı. Ardından olasılık tabanlı yöntemlerin hepsinin sağlaması gereken prefix özelliği tanımlandı: hiçbir kod başka bir kodun ön eki olamaz. Code1/Code2 örneği bu şartın neden zorunlu olduğunu gösterdi.

Sonra formülle üretilen statik prefix kodlar incelendi: logaritmik prefix kod, unary kod ve start-step-stop tripletleri (3,2,9 ile 680 kod), C_1-C_4 kodları ve Golomb kodu. Bu kodların farklı n değerlerinde farklı uzunluklar verdiği, seçimin olasılık dağılımına göre yapılacağı vurgulandı. Statik, yarı statik (semi-adaptif) ve adaptif yöntem ayrımından sonra ilk semi-adaptif yöntem olan Shannon-Fano iki örnekle işlendi; bölme yerinin kodları ve ortalama uzunluğu değiştirebileceği, entropiyi değiştirmeyeceği gösterildi. Ders, Huffman kodlamanın tanıtımı ve aynı olasılıklar için iki farklı ağaçla aynı ortalama uzunluğun elde edilmesiyle bitti. Hoca sonraki derste iki yol arasındaki seçimi varyans konusuyla anlatacağını, ardından dinamik Huffman, canonical Huffman (header'ı kısaltmak için) ve aritmetik kodlamaya geçeceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.